Reviewer Pack — Minimal Rank of Geometric Quantization over Read-Twice Branc…

Entry 768479185b7f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 04:22:02 UTC

Minimal Rank of Geometric Quantization over Read-Twice Branching Program Width

Entry ID: `768479185b7f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 04:22:02 UTC

1. Conjecture

Field A (mathematical branch): Geometric Quantization **Field B** (complexity object): Complexity Theory: Read-Twice Branching Programs

Statement:

> {'quantitative_relation': 'For all instances F of size n ≤ 40, the minimal rank of the geometric quantization matrix associated with F is Θ(log(n/2)) where F computes a function in P with read-twice branching program width w(F).', 'falsifier': 'There exists an instance F of size n ≤ 40 such that the minimal rank of the geometric quantization matrix associated with F is O(n) and F computes a function in P with read-twice branching program width w(F) = O(log n).'}

Rationale (proposer's reasoning):

> {'exposure_of_structure': 'Geometric quantization can provide insights into the symmetries and hidden structures of functions computed by complexity classes. If the rank is related to the branching program width, it suggests a deep connection between geometric properties and computational complexity.', 'natural_proofs_barrier_avoidance': 'This conjecture avoids the natural proofs barrier because it concerns the geometric quantization matrix, which cannot be easily transformed into a computable structure over a polynomial extension of the Boolean ring.'}

Taxonomy category: `ACC_SIPSER` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `60eadc94dd7f4f65`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The minimal rank of the geometric quantization matrix for an instance F is supported if it is between Θ(log(n/2)) and O(log n), where n ≤ 40 and F computes a function in P with read-twice branching program width w(F). It is falsified if any instance has a minimal rank that is O(n) or exceeds the upper bound of O(log n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - geometric quantization AND read-twice branching programs - quantum complexity AND minimal rank geometric quantization - branching program width AND θ(log(n/2)) geometric quantization

Top relevant hits considered: - [http://arxiv.org/abs/2403.13102v1] Geometric methods in quantum information and entanglement variational principle - [http://arxiv.org/abs/2005.11899v3] Geometric triangulations and highly twisted links - [http://arxiv.org/abs/math/0110001v3] A geometric interpretation of Milnor's triple linking numbers - [http://arxiv.org/abs/2410.04274v3] Bosonic Quantum Computational Complexity - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/2504.12989v3] Query Complexity of Classical and Quantum Channel Discrimination - [http://arxiv.org/abs/1806.04256v1] Pseudorandom Generators for Width-3 Branching Programs - [http://arxiv.org/abs/1709.08890v2] Partial matching width and its application to lower bounds for branching programs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_function(n):
        # Generate a random function in P with read-twice branching program width n
        return [random.choice([0, 1]) for _ in range(2**n)]

    def geometric_quantization_matrix(f):
        # Compute the geometric quantization matrix for a given function f
        m = len(f)
        n = int(math.log(m, 2))
        Q = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(m):
            x = bin(i)[2:].zfill(n)
            for j in range(n + 1):
                if f[i] == 0:
                    Q[j][j] += 1
```

```
            else:
                Q[0][j] += 1
    return Q

def min_rank(matrix):
    # Compute the minimal rank of a matrix using Gaussian elimination
    m = len(matrix)
    n = len(matrix[0])
    A = [row[:] for row in matrix]
    rank = 0
    for i in range(m):
        if all(A[i][j] == 0 for j in range(n)):
            continue
        rank += 1
        pivot_col = next(j for j in range(n) if A[i][j] != 0)
        for j in range(i + 1, m):
            factor = A[j][pivot_col] / A[i][pivot_col]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return rank

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = generate_random_function(n)
    Q = geometric_quantization_matrix(f)
    rank = min_rank(Q)
    results.append({
        "n": n,
        "rank": rank
    })

if not results:
    return {
        "metric_name": "min_rank",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

min_rank_values = [result["rank"] for result in results]
avg_rank = sum(min_rank_values) / len(min_rank_values)
std_dev = math.sqrt(sum((x - avg_rank) ** 2 for x in min_rank_values) / len(min_rank_values))

if any(rank > n * math.log(n, 2) for rank, n in zip(min_rank_values, n_values)):
    return {
        "metric_name": "min_rank",
        "metric_value": avg_rank,
        "instances_tested": len(results),
        "conjecture_holds": False,
        "counterexample": "FALSIFIED counterexample=rank_exceeds_bound first_failing_seed=<see
    }

return {
```

```
        "metric_name": "min_rank",
        "metric_value": avg_rank,
        "instances_tested": len(results),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(2, 6)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    avg_rank = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - avg_rank) ** 2 for result in results) / len(
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={avg_rank} std={std_dev} support_fraction={support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_rank} std={std_dev} support_fraction={support_fraction
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=rank_exceeds_bound first_failing_seed={first_fail
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, making it impossible to verify the conjecture's conditions. | next: Retry the test with a longer timeout or an alternative method that can produce results within a reasonable time frame.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12545
2	propose	ollama_remote	glm4:latest	0	0	10964
3	preregistration	ollama_remote	glm4:latest	0	0	5981
4	novelty	ollama_remote	glm4:latest	0	0	4622
5	novelty	ollama_remote	glm4:latest	0	0	8761
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	47191
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11060
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12514
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12800
10	judge	ollama_remote	glm4:latest	0	0	27881

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 154319 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/768479185b7f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/768479185b7f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/768479185b7f.tar.gz` (if generated)